

AI Landing Page Generation — All Prompts

Extracted from the Python FastAPI backend at D:\Universell Projects\universell-ai-apis. Scope: only the prompts that drive /api/generate-landing-page and /api/edit-landing-page (the AI Landing Page Generator feature). FastShop theme-generator workflow node prompts are out of scope.

There are **two parallel paths** through the landing page generator. A single environment variable picks which one runs.

Env var	Path	Where the prompts live
LANDING_USE_DESIGN_SYSTEM=false (default — legacy path)	One LLM call generates a full HTML page	app/landing_page/signature.py, app/landing_page/generator.py
LANDING_USE_DESIGN_SYSTEM=true (new — Design Intelligence System)	Curator picks a design pack → Composer fills each section in parallel → Jinja renderer composes HTML	app/design_system/ai/signatures.py, app/design_system/ai/curator.py, app/design_system/ai/composer.py

Both paths share the **image-prompt stage** (Stage 1) and the **edit endpoint** prompt.

0. Pipeline overview

POST /api/generate-landing-page (SSE)

- Stage 1: Image prompts — LandingPageImagePromptSignature (DSPy, 1 LLM call)
- Stage 2: Generate 3 images — DALL-E 3 / gpt-image-1-mini (no LLM prompt – uses Stage 1 output verbatim)
- Stage 3: HTML generation
 - (default) LandingPageHTMLSignature (DSPy, 1 LLM call → full HTML)
 - (new flag) DesignCuratorSignature → Nx SectionComposerSignature in parallel → Jinja render

POST /api/edit-landing-page — LandingPageEditSignature (DSPy, 1 LLM call, no SSE)

LLM: Azure OpenAI deployment gpt-5.1-codex-mini (reasoning model — responses.create API only). Image gen: gpt-image-1-mini (Azure).

1. STAGE 1 — Image-prompt generation

File: app/landing_page/signature.py **Class:** LandingPageImagePromptSignature

Module: dspy.ChainOfThought(LandingPageImagePromptSignature)

System / instruction text (DSPy signature docstring)

Generate 3 high-quality image prompts for a landing page.

You are an expert at creating detailed, photorealistic image generation prompts for DALL-E 3. Create exactly 3 prompts for different sections of a landing page.

RULES:

- NO text, letters, numbers, words, logos, or watermarks in the images
- NO UI elements, buttons, icons, or overlays
- NO mockups, screenshots, or device frames
- Images should be photorealistic, professional, and high-quality
- Each prompt should match the business type and design style
- Use cinematic lighting and professional photography style
- Prompts should complement the color palette specified

Input fields (all passed to the LLM)

Field	Description
description	Business description for the landing page
design_style	Design style: modern, minimal, bold, elegant, or playful
page_type	Page type: general, saas, portfolio, restaurant, ecommerce, service
primary_color	Primary brand color hex code (e.g., #2563EB)
secondary_color	Secondary brand color hex code (e.g., #1E40AF)

Output fields (each becomes the prompt for one image)

Field	Description
hero_prompt	DALL-E 3 prompt for the hero/banner background image. Should be wide, cinematic, and atmospheric. 1-2 sentences.
feature_prompt	DALL-E 3 prompt for the features/services section background or accent image. 1-2 sentences.
about_prompt	DALL-E 3 prompt for the about/testimonial section background or accent image. 1-2 sentences.

Fallback prompts (used only if the LLM call fails)

Defined in `app/landing_page/generator.py::_generate_image_prompts`:

```
hero    → "Professional hero banner for a {page_type} business.
{description[:100]}. Cinematic photography, warm lighting, no text."
feature → "Clean professional image showing services for a {page_type}
business. Modern, minimal, no text or UI elements."
about   → "Professional lifestyle image for a {page_type} business. Natural
lighting, candid atmosphere, no text."
```

2. STAGE 3 — HTML generation (DEFAULT / LEGACY PATH)

`LANDING_USE_DESIGN_SYSTEM=false`. One LLM call produces the entire HTML document.

2a. Per-page-type instruction block

File: `app/landing_page/generator.py` **Constant:** `PAGE_PROMPTS` (dict)

The frontend passes a `page_type` slug. The generator looks it up in this dict and **prepends the value** as `page_instructions` to the LLM call. The `page_type` slug also goes into a separate `page_type` input field.

```
PAGE_PROMPTS = {
    "homepage": (
        "Create a compelling HOMEPAGE with: a hero section (headline,
        subheadline, primary CTA button), "
        "a features/highlights section (3-4 key selling points with Font
        Awesome icons in a grid), "
        "a featured products or services section, social proof (testimonials
        or stats), "
        "and a CTA section. Tailor all copy to the business description
        provided."
    ),
    "shop": (
        "Create a PRODUCT CATALOG page with: a page header banner with title
        'Our Products' and a short tagline, "
        "a filter/category navigation bar (e.g. All, Coffee, Tea, Pastries –
        styled as pill buttons), "
        "a product grid layout with 6 product cards (each card has: product
        image placeholder using the provided image URLs, "
        "product name, short description, price like '$12.99', and an 'Add to
        Cart' button), "
        "and a 'Load More' button at the bottom. Use CSS grid with
        repeat(auto-fit, minmax(280px, 1fr)). "
        "Do NOT include hero sections, testimonials, about sections, or
        generic landing page content. "
        "This is a PRODUCT CATALOG page, not a homepage."
    ),
}
```

```
"services": (  
  "Create a SERVICES page with: a page header banner with title 'Our  
Services', "  
  "3-5 service cards in a grid (each card has: Font Awesome icon,  
service title, description, price, "  
  "and a 'Book Now' or 'Learn More' button), a process/how-it-works  
section with 3-4 numbered steps, "  
  "testimonials from service clients, and an enquiry/booking CTA  
section. "  
  "Do NOT include hero sections with generic landing page content. This  
is a SERVICES page."  
"bookings": (  
  "Create a BOOKING page with: a page header banner with title 'Book an  
Appointment', "  
  "a service selection section (3-4 selectable service cards with name,  
duration, and price), "  
  "a calendar/availability placeholder section (styled as a monthly  
calendar grid), "  
  "a customer details form (name, email, phone, preferred date/time,  
notes textarea), "  
  "and a confirmation/submit button. Do NOT include generic homepage  
content. This is a BOOKING page."  
"about": (  
  "Create an ABOUT US page with: a page header banner with title 'About  
Us', "  
  "a brand story section (2-3 paragraphs with an image on the side  
using CSS grid 2-column layout), "  
  "a mission and values section (3-4 value cards with icons), "  
  "a team introduction section (3 team member cards with circular image  
placeholders, names, and roles), "  
  "and a closing CTA section. Do NOT include product grids or generic  
homepage content. This is an ABOUT page."  
"contact": (  
  "Create a CONTACT page with: a page header banner with title 'Get In  
Touch', "  
  "a two-column layout: left column has a contact form (name, email,  
phone, subject, message textarea, submit button), "  
  "right column has business details (address with map icon  
placeholder, phone number, email, business hours), "  
  "social media icon links, and a map placeholder section (gray box  
with 'Map' text). "  
  "Do NOT include generic homepage content. This is a CONTACT page."  
"general": (  
  "Create a compelling landing page with: a hero section (headline,  
subheadline, CTA button), "  
  "a features section (3-4 cards with icons), an about section (2-
```

```

column layout with image), "
    "testimonials (3 cards), a contact section, and a footer. "
    "Tailor all copy to the business description."
),
}

```

The final assembled page_instructions string sent to the LLM is:

PAGE TYPE: {page_type}

{PAGE_PROMPTS[page_type]}

(optionally appended:)

Additional instructions from the user: {custom_prompt}

2b. HTML generation signature

File: app/landing_page/signature.py **Class:** LandingPageHTMLSignature **Module:** dspy.ChainOfThought(LandingPageHTMLSignature)

System / instruction text (DSPy signature docstring)

Generate a complete, production-ready single page as HTML with embedded CSS.

You are a world-class web designer. Create a stunning, modern, responsive page.

The output must be a COMPLETE HTML document with all CSS embedded in a <style> tag.

No external dependencies except Google Fonts and Font Awesome CDN.

CRITICAL: You MUST follow the page_instructions field EXACTLY. It tells you what type of page to build (e.g. homepage, product catalog, services page, booking page, about page, contact page). Do NOT always build a generic homepage. Build ONLY the page type requested.

NAVIGATION BAR (always include) - MUST use these EXACT CSS rules:

```

nav { position: fixed; top: 0; left: 0; width: 100%; z-index: 1000;
background: #ffffff;
    display: flex; align-items: center; justify-content: space-between;
padding: 15px 40px; box-shadow: 0 2px 10px rgba(0,0,0,0.08); }
nav .logo { font-size: 1.5rem; font-weight: 700; color: PRIMARY_COLOR;
text-decoration: none; }
nav .nav-links { display: flex; gap: 25px; align-items: center; }
nav .nav-links a { color: #333; text-decoration: none; font-weight: 500;
font-size: 0.95rem; transition: color 0.3s; }
nav .nav-links a:hover { color: PRIMARY_COLOR; }
@media(max-width:768px) { nav { padding: 12px 20px; } nav .nav-links {
gap: 15px; } nav .nav-links a { font-size: 0.85rem; } }
HTML structure: <nav><a href="#" class="logo">BRAND</a><div class="nav-

```

```
links">...</div></nav>
```

IMPORTANT: The first section MUST have padding-top: 80px to prevent content from hiding behind the fixed nav.

FOOTER (always include) - dark background (#222), centered text, social icon links using Font Awesome.

PAGE CONTENT: The page_instructions field tells you EXACTLY what sections to build.

Follow it precisely. Do NOT default to a generic homepage layout.

IMAGE RULES:

- Use the provided image URLs as CSS background-images or tags as appropriate
- ALL images must have: max-width set, object-fit: cover, and a fixed or max height
- Do NOT let any image overflow its container or stretch beyond the viewport width
- NEVER use tags without explicit width, height, max-width, and object-fit CSS

LAYOUT RULES:

- Every section must have: overflow: hidden; on the section element
- Use max-width: 1200px and margin: 0 auto for all content containers
- The body must have: margin: 0; padding: 0; overflow-x: hidden;
- All sections must have proper padding (60px-100px vertical, 20px horizontal)
- Cards: use CSS grid with grid-template-columns: repeat(auto-fit, minmax(280px, 1fr))
- Hero overlay MUST use a dark semi-transparent gradient for readability
- ALL text on image backgrounds MUST be white (#ffffff) for contrast
- Primary color usage: buttons background, icon colors, logo text, card accents, link hover
- Section headings on white/light backgrounds: use #222222 or #333333
- Navigation bar: white background (#ffffff), logo in primary color, links in #333
- Smooth scroll:

```
html { scroll-behavior: smooth; }
```

CSS RULES:

- CSS reset:

```
* { margin: 0; padding: 0; box-sizing: border-box; }
```
- Import ONE Google Font via @import at the top of the style tag
- Use Font Awesome 6 CDN link in the head for icons
- Use the exact primary_color and secondary_color hex values provided
- All buttons: padding 14px 32px, border-radius 8px, font-weight 600, cursor pointer
- Cards: background white, border-radius 12px, box-shadow 0 4px 20px rgba(0,0,0,0.08), padding 30px
- Responsive: @media(max-width:768px) stack columns, reduce font sizes, 20px padding

- All hover effects must use transition: all 0.3s ease

OUTPUT RULES:

- Output ONLY valid HTML starting with `<!DOCTYPE html>`
- No markdown, no explanation, no code fences, no comments outside the HTML
- All CSS in a single `<style>` tag in the `<head>`
- The page must look polished, professional, and modern

Input fields

Field	Description
description	Business description
design_style	modern, minimal, bold, elegant, or playful
page_type	homepage, shop, services, bookings, about, contact, general
page_instructions	CRITICAL. The assembled per-page block from PAGE_PROMPTS above (+ optional user custom_prompt).
primary_color	Hex code
secondary_color	Hex code
image_urls	JSON string: {"hero": "url", "feature": "url", "about": "url"}

Output

Field	Description
html	Complete HTML document. Must start with <code><!DOCTYPE html></code> and end with <code></html></code> . No markdown fences.

3. STAGE 3 — HTML generation (NEW / DESIGN INTELLIGENCE PATH)

LANDING_USE_DESIGN_SYSTEM=true. Two LLM stages:

1. **Curator** (one call) → picks theme, fontPack, colorPalette, animationPack, voice, sectionPlan
2. **Composer** (one call per section, run in parallel) → fills the content slots of each section

Jinja renders the final HTML from the picked design pack + per-section content. No HTML generation by the LLM in this path.

3a. Design Curator signature

File: app/design_system/ai/signatures.py **Class:** DesignCuratorSignature

Select the best design pack for a business landing page.

You are a senior product designer. Your job is to pick a curated design package – theme, font pack, color palette, and animation pack – that produces a stunning, modern, premium landing page for the business.

HARD RULES:

- Output keys MUST be drawn EXACTLY from the available_* lists. Do not invent new keys, do not adjust spelling, do not return descriptions.
- Match the design to the business:
 - AI/SaaS / dev tools → modern_saas or ai_futuristic
 - Luxury fashion / jewelry / general luxury → luxury_brand
 - BEAUTY / makeup artist / hair stylist / wedding / spa / salon / hospitality → luxury_beauty (cream) or luxury_beauty_dark (black). These industries need photo-driven editorial design.
 - Finance / B2B / consulting → enterprise_corporate
 - Bold startups / crypto → startup_dark
 - Studios / agencies / portfolios → minimal_clean.
- The user's primary_color_hint is informational – pick the palette whose vibe matches it, not necessarily an identical hue.
- The voice should match the audience: B2B → professional or trustworthy, consumer/lifestyle → playful or energetic, premium brand → luxurious.
- INDUSTRY DETECTION for sectionPlan: scan the description for these signals and pick the matching plan shape:
 - "makeup artist", "hair stylist", "beauty", "salon", "wedding", "spa", "esthetician", "lash", "brow", "bridal" → editorial/photo plan: ["navbar.glass", "hero.editorial", "services.photo_grid", "gallery.instagram", "quote.featured", "faq.accordion", "booking.simple_cta", "footer.startup"]
 - Other service businesses (coaching, consulting, freelance) → conversion plan with hero.cinematic + features.bento_grid + service.pricing as before.
- Keep rationale to ONE short sentence (≤ 20 words).

Curator input fields

Field	Description
description	Business description from the user
page_type	New 8-type taxonomy: product-launch, lead-generation, event-webinar, portfolio, service-showcase, coming-soon, discount-promo, about-brand-story. Legacy slugs (homepage, shop, services, bookings, about, contact, general) also supported.
design_style	modern, minimal, bold, elegant, or playful
custom_prompt	Optional extra direction from the user (may be empty)

Field	Description
<code>primary_color_hint</code>	User's preferred primary color hex (informational hint, may be empty)
<code>available_themes</code>	JSON list of available themes with key + description
<code>available_font_packs</code>	JSON list of font pack keys
<code>available_palettes</code>	JSON list of palettes with key + light/dark mode
<code>available_animation_packs</code>	JSON list of animation pack keys
<code>available_sections</code>	JSON list of available sections grouped by category, with key + tags + description

Curator output fields

Field	Description
<code>theme</code>	One key from <code>available_themes</code> — must match exactly
<code>fontPack</code>	One key from <code>available_font_packs</code> — must match exactly
<code>colorPalette</code>	One key from <code>available_palettes</code> — must match exactly
<code>animationPack</code>	One key from <code>available_animation_packs</code> — must match exactly
<code>voice</code>	One of professional / playful / luxurious / energetic / calm / trustworthy
<code>sectionPlan</code>	JSON array of 5–9 section type keys, in display order. Must start with <code>navbar.*</code> and end with <code>footer.*</code> . Must include exactly one <code>hero.*</code> . See per-page-type guidance below.
<code>rationale</code>	One short sentence (≤ 20 words) explaining the design choice

Embedded page-type guidance for `sectionPlan` (inside the `OutputField` description)

PAGE-TYPE-AWARE GUIDANCE — pick sections matching the user's intent:

- product-launch → `hero.image_overlay` or `hero.cinematic` + `product.showcase` + `features.bento_grid` + `about.split_image`.
- lead-generation → `hero.lead_form` (has embedded form) + `features.bento_grid` (offer benefits) + `faq.accordion`. NO testimonials.
- event-webinar → `hero.event_countdown` + `event.details` + `event.agenda` + (optional) `event.speakers` + `faq.accordion`. NEVER include `testimonials.marquee` — events haven't happened yet.
- portfolio → `hero.cinematic` + `portfolio.gallery` + `about.split_image` + `faq.accordion`.
- service-showcase → `hero.cinematic` + `features.bento_grid` + `service.pricing` +

about.split_image + faq.accordion.

- coming-soon → hero.coming_soon + features.bento_grid (what's coming) + faq.accordion. NO testimonials.
- discount-promo → hero.promo + product.showcase + cta.glowing + faq.accordion. NO testimonials.
- about-brand-story → hero.cinematic + about.split_image + brand.values + brand.timeline + cta.glowing.

Example:

```
["navbar.glass", "hero.event_countdown", "event.details", "event.agenda", "faq.accordion", "cta.glowing", "footer.startup"]
```

Python-side safety net (industry override in curator.py)

If the AI fails to follow the BEAUTY industry detection rule, a keyword-based Python check overrides the plan when `page_type ∈ {service-showcase, service_showcase, services, bookings}` AND the description contains beauty/wedding/spa keywords:

```
_BEAUTY_INDUSTRY_SIGNALS = (  
    "makeup", "make-up", "make up",  
    "hair stylist", "hairstylist", "hair artist", "hair salon",  
    "beauty salon", "beauty studio", "beauty artist",  
    "esthetician", "esthetics", "aesthetic",  
    "wedding", "bridal", "bride",  
    "spa", "lash", "brow", "nail", "manicur", "pedicur",  
    "skincare", "skin care", "facial", "massage therap",  
    "barber", "blowout", "balayage",  
    "photographer", "videographer", "boudoir",  
)
```

If matched → forces editorial plan + `luxury_beauty` (or `luxury_beauty_dark` if description mentions “dark theme”, “black theme”, “high fashion”, “editorial dark”, “dark luxury”) + `editorial_luxe` fonts + `premium_luxe` animation + luxurious voice.

Default section plans (used if Curator output is invalid)

Defined in `app/design_system/ai/curator.py::_default_section_plan`. These represent the **canonical layout per page type** and are the fallback safety net:

```
product-launch → [navbar.glass, hero.image_overlay, product.showcase,  
features.bento_grid, about.split_image, cta.glowing, footer.startup]  
lead-generation → [navbar.glass, hero.lead_form, features.bento_grid,  
about.split_image, faq.accordion, footer.startup]  
event-webinar → [navbar.glass, hero.event_countdown, event.details,  
event.agenda, event.speakers, faq.accordion, cta.glowing, footer.startup]  
portfolio → [navbar.glass, hero.cinematic, portfolio.gallery,  
about.split_image, faq.accordion, cta.glowing, footer.startup]  
service-showcase → [navbar.glass, hero.cinematic, features.bento_grid,  
service.pricing, about.split_image, faq.accordion, cta.glowing,  
footer.startup]  
service-showcase-editorial (beauty override) → [navbar.glass, hero.editorial,
```

```

services.photo_grid, gallery.instagram, quote.featured, faq.accordion,
booking.simple_cta, footer.startup]
coming-soon      → [navbar.glass, hero.coming_soon, features.bento_grid,
faq.accordion, footer.startup]
discount-promo   → [navbar.glass, hero.promo, product.showcase,
features.bento_grid, faq.accordion, footer.startup]
about-brand-story→ [navbar.glass, hero.cinematic, about.split_image,
brand.values, brand.timeline, cta.glowing, footer.startup]
contact          → [navbar.glass, hero.ai_saas, cta.glowing, footer.startup]
homepage/general → [navbar.glass, hero.cinematic, features.bento_grid,
about.split_image, testimonials.marquee, faq.accordion, cta.glowing,
footer.startup]

```

3b. Section Composer signature

File: app/design_system/ai/signatures.py **Class:** SectionComposerSignature

Runs **once per section** in sectionPlan, in parallel via asyncio.gather. Each call writes the copy for one section against its JSON schema.

Write content for ONE section of a landing page, conforming to its schema.

You are a world-class conversion copywriter. Given a section type and its content schema (which slots to fill, with maximum lengths), produce JSON that matches the schema exactly.

HARD RULES:

- Output JSON ONLY in content_json. No prose, no markdown fences, no comments. Start with `{`, end with `}`.
- Fill EVERY required slot. Skip optional slots if you do not have strong content for them – never invent placeholder text like "Lorem ipsum" or "Coming soon".
- Respect maximum lengths strictly. Trim before truncating mid-word.
- Copy must be SPECIFIC to the business description. No generic phrases like "We deliver excellent service" or "Trusted by leading brands".
- For icon fields, use Font Awesome 6 class names WITH the prefix style (e.g. "fa-bolt", "fa-shield-halved", "fa-rocket", "fa-calendar-days", "fa-microphone", "fa-location-dot"). Pick icons relevant to each item. Never return empty strings or invented icon names.

ANTI-FABRICATION (these are MANDATORY – violations destroy user trust):

- Never invent statistics, prices, customer counts, ratings, or revenue figures. If a number is not in the description, omit the field.
- Never invent testimonials, attendee names, customer quotes, or speaker names. If the description contains zero real testimonials/speakers, return an EMPTY items[] array – the renderer will skip the section.
- NEVER recycle the brand name, tagline, or business description as a "testimonial quote", a "service name", or a "FAQ question/answer". Those exist as their own fields elsewhere in the page. If the description doesn't contain a REAL service/quote/FAQ as distinct

content, return an EMPTY items[] (or empty string for `quote`) – the renderer will skip the section. A page with one solid hero is better than a page padded with brand-name-as-service-name cards.

- Never invent URLs, image links, email addresses, phone numbers, street addresses, or registration links. Only use values that appear LITERALLY in the description. Phone/address/email belongs in footer/contact slots ONLY if explicitly provided.
- For HERO subheadlines: write a marketing hook ONLY. Never include phone numbers, email addresses, street addresses, postal codes, or formatted date/time strings ("2026-05-31 at 19:34") – those go in dedicated sections (event.details, footer.startup) which have proper slots for them. The hero subheadline is one or two sentences of value prop, not a data dump.
- HTML is forbidden inside text fields. If the description contains `<img src='...'>` tags, extract the URL into the matching imageUrl slot (NOT into description text). Strip every HTML tag from text fields.

ARRAYS:

- For array slots with minItems > 0, you must produce at least minItems ONLY if you have real source content for them. If the description does not justify any items, return [] and rely on the renderer to skip the section. Never fabricate items just to hit minItems.
- Only add more than the minimum if you have substantive distinct content.

Composer input fields

Field	Description
description	Business description — copy MUST be specific to this
page_type	Page type slug for context
section_type	Section type key (e.g. hero.ai_saas, cta.glowing)
schema_spec	JSON spec describing required slots, types, and max lengths (pulled from SECTION_SCHEMAS in app/design_system/sections/schemas.py)
voice	Tone of voice chosen by the curator (professional / playful / luxurious / energetic / calm / trustworthy)
custom_prompt	Optional extra direction from the user (may be empty)

Composer output

Field	Description
content_json	JSON object matching the schema_spec EXACTLY. Must parse with json.loads().

4. EDIT — /api/edit-landing-page

File: app/main.py (around line 1579) **Inline class:** LandingPageEditSignature (declared inside the endpoint) **Module:** dspy.ChainOfThought(LandingPageEditSignature)

Signature docstring (used as the LLM instruction)

Edit an existing HTML landing page based on a user request.
Return the COMPLETE updated HTML document preserving all existing styles and content.
Only modify what the edit request specifically asks for.

edit_instructions field value (Python f-string sent to the LLM each call)

This is the full operational prompt — built per request and passed in as the edit_instructions input field. {request.edit_request} is the user's natural-language change.

You are editing an existing HTML landing page. The user wants you to make this change:

EDIT REQUEST: {request.edit_request}

RULES:

- Return the COMPLETE updated HTML document (<!DOCTYPE html> to </html>).
- ONLY change what the edit request asks for. Do NOT rewrite or remove other content.
- Preserve ALL existing CSS styles, classes, IDs, images, links, and structure.
- Keep all <style> tags with their full CSS intact – copy them EXACTLY.
- Keep all <link> tags (Google Fonts, Font Awesome CDN) in <head>.
- Keep ALL existing image URLs (src attributes and background-image URLs) EXACTLY as they are.
- Do NOT invent, fabricate, or change image file paths. Never use paths like /landing_hero_xxx.png.
- If the user asks to ADD a new image and you need a placeholder, use:
https://placeholder.co/1200x600/333/fff?text=Hero+Image
or https://picsum.photos/1200/600 for a random photo. NEVER use local file paths for new images.
- For background images, use: background-image: url('https://picsum.photos/1920/800'); background-size: cover;
- Do NOT strip, simplify, or minify the HTML/CSS.
- Output ONLY the HTML. No markdown fences, no explanation.

Input fields

Field	Description
current_html	The current full HTML document of the landing page
edit_instructions	The block above, with the user's edit_request substituted in

Output

Field	Description
html	The complete updated HTML document. Must start with <code><!DOCTYPE html></code> . Preserve ALL existing CSS and content that was not explicitly changed.

5. Quick reference — every prompt at a glance

#	Stage / Endpoint	Prompt source (file → name)	Path
1	Stage 1 — image prompts	app/landing_page/signature.py → LandingPageImagePromptSignature	Both
2	Stage 3 — HTML (legacy)	app/landing_page/generator.py → PAGE_PROMPTS dict	Legacy only
3	Stage 3 — HTML (legacy)	app/landing_page/signature.py → LandingPageHTMLSignature	Legacy only
4	Stage 3a — design pack	app/design_system/ai/signatures.py → DesignCuratorSignature	New only
5	Stage 3b — section copy	app/design_system/ai/signatures.py → SectionComposerSignature	New only
6	Edit endpoint	app/main.py (inline) → LandingPageEditSignature + inline edit_prompt f-string	Both

Generated from the Python backend on 2026-05-13. If any prompt has been edited after this date, re-run the extraction.